

PERMUTACIONES USANDO OPERACIONES BIT A BIT

► **Areli Alejandra Guevara Badillo**

► viernes, 7 de noviembre de 2025

Instrucciones

Diseña e implementa en lenguaje de programación C una función que reciba como entrada una permutación P de tamaño 8 y un dato en variable de tipo unsigned char.

La función debe aplicar la permutación P a los bits del dato y regresar el dato con los bits permutados.

La función debe usar operaciones bit a bit tales como and, xor, or, etc.

Debe mostrar como salida el resultado en hexadecimal y en binario.

Desarrollo

```
1 // Función para permutar los bits según la tabla P
2 unsigned char permutarBits(unsigned char dato, int P[]) {
3     unsigned char permutado = 0;
4
5     for(int i = 0; i < 8; i++) {
6         unsigned char bit = (dato >> (8 - P[i])) & 1;
7         permutado |= (bit << (7 - i));
8     }
9     return permutado;
10 }
```

La función implementada recibe como entradas el dato de tipo *unsigned char* y un arreglo P que contiene la permutación.

La permutación se realiza de la siguiente forma:

1. Inicializa permutado (que es la variable donde guardaremos el resultado de la permutación) en 0.
2. Ingresa a un for que va de 0 a 7 que busca recorrer todo el arreglo P y recorrer bit a bit el dato.
 - a. Dentro del for primero recupera el bit de interés (que es el que está en la posición que se va a permutar) de la siguiente forma:
 - Se realiza un desplazamiento a la derecha de 8 menos la posición del bit de interés, esto dejara el bit deseado en el bit menos significativo.
 - Después se realiza un AND con 1 para quedarnos únicamente con el bit que nos interesa.

- b. Después se agrega el bit de interés al resultado *permutado* de la siguiente forma:
- Se realiza un desplazamiento a la izquierda al bit de 7 menos el índice que maneja nuestro for, esto para que quede en la posición que nos marca la permutación.
 - Después se hace un OR con *permutado* para agregar el bit a nuestro resultado y se guarda en *permutado*.

3. Finalmente devuelve la variable *permutado* que contiene nuestro resultado de la permutación.

Para imprimir los resultados en los formatos requeridos se utilizaron las siguientes líneas de código:

```
1 for(int i = 7; i ≥ 0; i--) {  
2     printf("%d", (permutado >> i) & 1);  
3     if(i == 4) printf(" ");  
4 }  
5 printf("\n0x%02X\n", permutado);
```

Para imprimir el resultado en binario se realizó un corrimiento dentro de un for que deja el bit de interés (en este caso va de bit a bit desde el más significativo en índice 7 al menos significativo en el índice 1), realiza un AND con 1 para quedarnos únicamente con el bit de interés y lo imprime en formato decimal (que pertenecerá a {0, 1} gracias al corrimiento y al AND que se realizaron).

Y para imprimir en formato hexadecimal únicamente se especificó el formato al momento de imprimir el resultado (%X da el formato hexadecimal).

Ejecución del código

```
Ingresa la permutacion [tamano 8]  
P[1] = 4  
P[2] = 3  
P[3] = 2  
P[4] = 1  
P[5] = 8  
P[6] = 7  
P[7] = 6  
P[8] = 5  
  
Ingresa un numero decimal: 66  
Dato original:  
0100 0010  
0x42  
  
Dato permutado:  
0010 0100  
0x24
```

Comprobación

dato = 66 → 0x42
0100 0010

P = ^{0 1 2 3 4 5 6 7}
4 3 2 1 ~~8~~ ~~9~~ 6 5 permutado = 0000 0000

	i	P[i]	8 - P[i]	dato >> 8[i]	bit & 1	7-i	bit << (7-i)	permutado i
1 ^{er} for:	0	4	4	<u>0000</u> 0100	0	7	<u>00000000</u>	<u>00000000</u>
2 ^{do} for:	1	3	5	<u>0000</u> 0010	0	6	<u>00000000</u>	<u>00000000</u>
3 ^{er} for:	2	2	6	<u>00000001</u>	1	5	<u>00100000</u>	<u>00100000</u>
4 ^{to} for:	3	1	7	<u>00000000</u>	0	4	<u>00000000</u>	<u>00100000</u>
5 ^{to} for:	4	8	0	0100 0010	0	3	<u>00000000</u>	<u>00100000</u>
6 ^{to} for:	5	7	1	<u>0010</u> 0001	1	2	<u>00000100</u>	<u>00100100</u>
7 ^{mo} for:	6	6	2	<u>0001</u> 0000	0	1	<u>00000000</u>	<u>00100100</u>
8 ^{vo} for:	7	5	3	<u>0001</u> 0100	0	0	<u>00000000</u>	<u>00100100</u>

permutado = 00100100 → 0x24
36